

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Florian Adamczyk^a, Felix Hollfoth^b, Kolja Scriba^c, Björn Becker^d and Felix Neumann^e

^aJustus-Liebig-Universität, Matrikelnr. 8105234

^bJustus Liebig Universität, Matrikelnr. 8221324

^cJustus Liebig Universität, Matrikelnr. 8111369

^dJustus Liebig Universität, Matrikelnr. 8206290

^eJustus Liebig Universität, Matrikelnr. 8116639

Gruppe 308 – KI I Projekt WS 2025/26

Zusammenfassung—Dieser Ergebnisbericht dokumentiert die Ergebnisse der Gruppe 308 der Projektarbeit zu Maschinellem Lernen im Modul Künstliche Intelligenz 1. Die Aufgabenstellung lautete auf dem Datensatz zu den kalifornischen Hauspreisen, wie in der Vorlesung betrachtet ein neuronales Netz, um die Hauspreise zu beschreiben zu entwickeln. Im Anschluss soll dieses mit der linearen Regression verglichen werden. Die Bearbeitungszeit betrug pro Gruppenmitglied mindestens 75 Stunden und wurde in mehreren Meetings zwischen den Personen aufgeteilt. Neben der linearen Regression wurden auch andere Regressionsmodelle als Vergleichsgrundlage für das Neuronale Netz herangezogen. Der Schwerpunkt liegt allerdings auf der sukzessiven Verbesserung des Neuronalen Netzes, unter Anwendung der Vorlesungsinhalte. Die Optimierung der Regression erfolgt unter ständiger Kontrolle der Fehlergrößen wie dem Mean Squared Error (=MSE) und dem Root Mean Square Error (=RMSE), sowie dem coefficient of determination ($= R^2$) als quantitativer Vergleich der Modelle.

Keywords—eurnales Netz, California Housing, TensorFlow, Regression, Machine Learning

1. EXPLORATIVE DATENANALYSE, -BEREINIGUNG UND -DARSTELLUNG (FLORIAN)

Die explorative Datenanalyse dient der Aufnahme von Eckdaten des Datensatzes und dem Erlangen eines Gefühls für potenzielle Besonderheiten, die in der Datenbereinigung für ein besseres Ergebnis sinnvoll behoben werden.

1.1. Analyse

Die Explorative Datenanalyse ergab, dass zwei der acht Features des Datensatzes cut-off-Werte besitzen. Eine unrealistisch hohe Anzahl an Werten befindet sich hierbei genau an der Grenze der aufgetragenen Werte. Zudem ist eine hohe Streuung der Werte mit einigen vermeintlichen Ausreißern direkt zu erkennen. Um damit überhaupt beginnen zu können, hat Florian ein Projektinfrastruktur mit git, allgemeinen Funktionen, Anweisungen und den utils geschaffen, sowie die Daten eingelesen. Auf dieser Grundlage konnte dann mit der inhaltlichen Arbeit begonnen werden.

1.2. Bereinigung

Die Cut-Off-Werte der Features 'MedHouseVal' und 'HouseAge' wurden mit den Outliern über dem 98%-Quantil bei den Features 'AveRooms', 'AveBedrms', 'Population' und 'AveOccup' entfernt, da diese Artefakte darstellen und das Rauschen des Datensatzes erhöhen. Aus den ursprünglich 20.000 Datenpunkten verbleiben nach der Bereinigung noch 17.386.

1.3. Skalierung

Zuletzt wurde der Datensatz noch in einen Test-(80% Anteil) und Trainingsdatensatz(20% Anteil) aufgeteilt. Aus dem Testdatensatz wurde dann zum Lernen der Modelle noch ein Validierungsdatensatz entnommen. Für das Training der Modelle mit verschiedensten Aktivierungsfunktionen ist eine Skalierung zwingend notwendig um ein sinnvolles Ergebnis zu erhalten. In Notebook 0c wurde zu diesem Zweck ein Standard-Skalierer und drei Minimum-Maximum-Skalierer implementiert, auf die in den jeweiligen Notebooks zurück gegriffen werden wird.

1.4. Darstellungen bereinigter Daten

Fig. 1 Datensatz mit entfernten Cut-Off-Werten und Ausreißern sortiert nach Features. Im Vergleich mit den Originaldaten gibt es bei den modifizierten Features keine gehäuften Datenansammlungen am Ende der Skala und keine groben Ausreißer mehr.

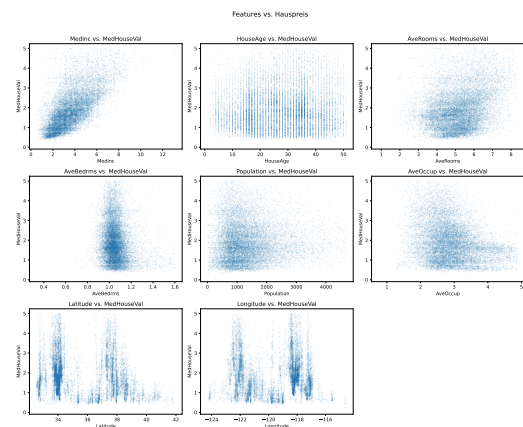


Abbildung 1. Bereinigte Daten sortiert nach Features.

Fig. 2 Die nach der Datenbereinigung durchgeführte Korrelationsanalyse zeigt den starken Zusammenhang zwischen 'MedHouseVal' und 'MedInc'.

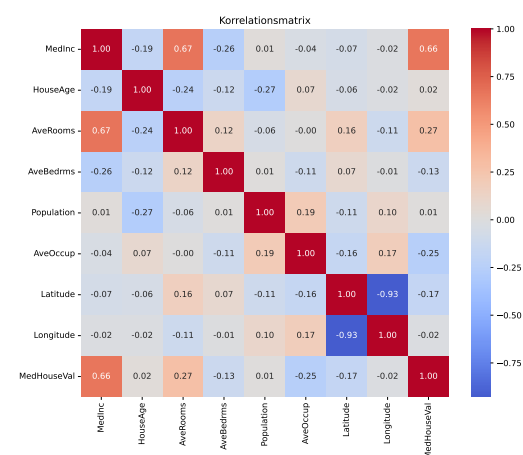


Abbildung 2. Korrelationsanalyse der bereinigten Daten

2. LINEARE REGRESSION

Die als Vergleichsmodell für die Bewertung komplexerer Modelle von Florian implementierte lineare Regression, erzielt unskaliert folgende Ergebnisse:

Tabelle 1. Ergebnisse der linearen Regression

Metrik	Train	Test
$R^2 - \text{Score}$	0.6465	0.6326
MAE	0.4252	0.4331
RMSE	0.5710	0.5779

Diese Werte stellen die "Baseline" für Einordnung komplexerer Modelle dar und geben einen ersten Anhaltspunkt über die potenzielle Leistung der Regressionen.

2.1. LASSO und Ridge (Florian, Kolja)

Eine Erweiterung der linearen Regression durch eine LASSO-Ridge Kombination, sowie die Verwendung des Standard-Scaler führten zunächst zu keiner Verbesserung gegenüber der Baseline. Entscheidende Unterschiede zeigten sich durch die Einführung der Polynomialen Erweiterung. Zunächst eine Verbesserung in der zweiten und nochmal geringfügig in der dritten Ordnung. Am Ende konnte ein $R^2 = 0.7184$ auf die Testdaten erreicht werden ($\Delta_{\text{Baseline}} R^2 = 8.58\%$). Durch die gewählten Methoden konnte auch ein sehr geringer Overfitting-Gap erzielt werden.

3. DECISION TREE REGRESSION (FLORIAN)

Ausgehend von den Vorlesungsinhalten wurden Decision Trees zunächst als Referenz ohne Einschränkungen und dann mit Beschränkung der maximalen Tiefe gegen Overfitting Probleme trainiert. Der Overfitting-Gap bei erstem lag bei knapp 45%. Bei zweitem wurde der bias-varianz-tradeoff sichtbar. Durch Pruning konnte das Overfitting des Baums auf die Testdaten sehr erfolgreich unterbunden werden. Wie auch bei der Explorativen Datenanalyse zeigt sich, dass das Median Einkommen (=MedInc) die Featureimportance dominiert (siehe Fig. 5).

4. ENSEMBLE (FLORIAN)

Die Ensemble-Methoden wurden als Test für die mögliche Genauigkeit der Regression verwendet. Die Ergebnisse der Decision Trees konnten als Random Forest enorm verbessert werden. Auch die Verwendung des Gradient Boosting und die Optimierung der learning rate konnten bessere Regressionen liefern. Diese Erkenntnis und ein maximales $R^2 = 0.81$ ($\Delta_{\text{Baseline}} R^2 = 17.74\%$), als Marker für die mögliche Leistung der Regression, wurden in das Training neuronaler Netze einbezogen.

5. K-NEAREST NEIGHBORS REGRESSION (FLORIAN)

Die K-Nearest Neighbors Regression hingegen zeigte keine allzu zuverlässigen Ergebnisse. Sie zeigte jedoch auf, wie unerlässlich eine Skalierung der Daten sein kann. Eine gleichzeitige Reduktion zweier Fehlerquellen führt zum bias-variance-tradeoff bei der Wahl des besten k. Durch Distance weighting wurde eine Verbesserung erzielt, allerdings rechtfertigen die Ergebnisse ($\Delta_{\text{Baseline}} R^2 \approx 3.01\%$) die viel längere Rechenzeit nicht.

6. NEURONALE NETZE

Verschiedene Neuronale Netze (=NN) wurden, ausgehend von einem sequenziellen Netz mit einem Hidden Layer (=HL) als Minimalnetz, zunächst ohne optimierte Parametrisierung, ausprobiert. Bereits bei diesem Minimalnetz ergab sich ($\Delta_{\text{Baseline}} R^2 \approx 10\%$). Im Anschluss testeten wir verschiedene Optimierungsansätze:

6.1. Architektur

Durch eine geschickte Wahl oder bewusstes Freistellen der Wahl der Parameter des Aufbaus des neuronalen Netzes wurden kleine Verbesserungen der Regression erreicht.

6.1.1. Layer (Björn, Felix H., Felix N., Florian; Kolja)

Die Variation der Breite und Layer ergab keine großen Veränderungen für das Neuronale Netz. Ein zu schmaler Layer vermag die komplexen Zusammenhänge zwischen einzelnen Merkmalen nicht ausreichend zu beschreiben. Zu viele Neuronen in einem Layer führten hingegen zu Overfitting. Dieses konnte durch die Verwendung eines Dropout-layers drastisch reduziert werden. Es stellte sich zudem heraus, dass obgleich bei isolierter Analyse der Tiefe an einfachen Netzen eine größere Tiefe den R^2 weiter steigen lässt, bevor bei ungefähr 10 Layern das overfitting übernimmt vgl. Fig. 3, dass bei komplexeren und breiteren Modellen jedoch keine große Tiefe für eine relativ genaue Regression notwendig ist. Diese erhöht nur zunehmend das Overfitting. Es wurden gute Ergebnisse mit zwei-drei Hidden Layers, einer [128 – 64 – 32]-Neuronenverteilung und einem Dropoutlayer, erzielt.

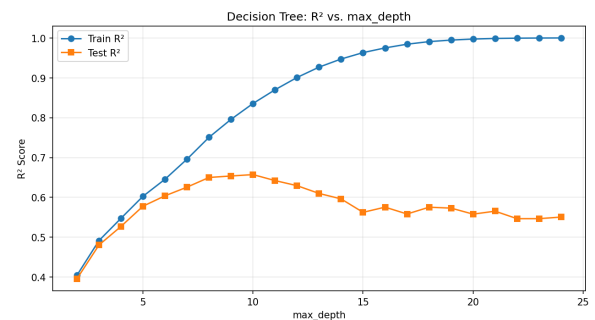


Abbildung 3. maximale Tiefe

6.1.2. Regularisierung (Kolja)

Untersuchungen des Einflusses einer L2-Regularisierung verglichen mit dem Basismodell ohne Regularisierung erzielte einen signifikant geringeres Overfitting-Gap (-40%). Damit verbessert sich die Generalisierungsfähigkeit der Modelle erheblich. Nach Analyse der L2-Regularisierung, des Dropout-Verfahrens und einer Kombination konnte ein von Modell zu Modell sukzessive verringertes Overfitting zulasten des R^2 festgestellt werden. Durch die drastische Verringerung des Overfittinggaps reduzierte sich dieser um einige Zehntel Prozent.

6.1.3. Aktivierungsfunktionen (Felix H., Kolja)

Ein Test der sechs gängigsten Aktivierungsfunktionen (ReLU, tanh, sigmoid, ELU, SELU, Leaky-ReLU) im Zusammenspiel mit den verschiedenen Skalierung zeigt die beste Leistung auf den Testdaten mit dem Tangens hyperbolicus angewandt auf den Standard-Scaler. Dieser erzeugt viele Daten um Null herum, die tangens hyperbolicus gut verarbeiten kann. Zudem neigt diese Funktion wegen ihrer Glätte zu wenig Overfitting. Spätere komplexere Modelle zeigen, dass die Leistung nur geringfügig von der Aktivierungsfunktion abhängt und die Wahl dem Modell überlassen werden kann.

6.1.4. Hyperparameter (Björn, Felix H., Felix N.)

Zunächst fand eine Analyse der Hyperparameter unter festhalten der restlichen Parameter statt, um eine Idee für die Größenordnung zu gewinnen. Die isolierte Hyperparameterbetrachtung endete dabei häufig in lokalen Minima ohne eindeutiges Optimum. Dennoch stellten sich folgende Größenordnungen als Anhaltspunkte für weitere Untersuchungen heraus.

Tabelle 2. Ergebnisse der Hyperparameteruntersuchung

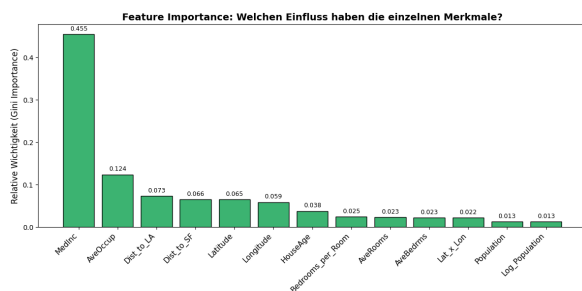
Hyperparameter	Größenordnung	Kommentar
Epochen	300-500	
learning-rate	0.01- 0.001	im Folgenden meist mit Lr-Scheduler. Dennoch fast immer in diesem Bereich
Batch-Size	16-128	größer würde zu einer zu langen Konvergenzzeit führen

Das Zusammenspiel der Hyperparameter wurde zunächst mit Random Search und zur höheren Recheneffizienz mit einer Bayesschen Optimierungsfunktion durchgeführt. Die Learningrateoptimierung erfolgt dynamisch während des Trainings mit einem Learningrate-Scheduler.

Iles in allem führt die händische Veränderung der Architektur, der Lernrate und der Epochen zunächst zwar zu kleinen Ergebnisschwankungen, ist aber ohne systematisches Vorgehen und Simultanoptimierung nicht zielführend. Die besten Modelle variieren daher in der Zusammensetzung der Architektur, liegen aber alle in den genannten Größenordnungen.

6.2. Feature engineering (Felix N., Björn, Felix H.)

Eine Untersuchung der Features zeigt eine deutliche Spanne an potenziellen Informationsgewinn durch die einzelnen Features. Daher wurde versucht die Modelle mit den wichtigsten vier bis fünf und auch nur dem wichtigsten Feature zu trainieren. Eine geringere Featureanzahl verringert die Komplexität des Modells und somit die Rechenzeit. Obgleich eine geringere Featureanzahl zu keine höheren Leistung führt, können Ensembles aus Modellen die verschieden viele Features berücksichtigen dies leisten. Um die Korrelation einzelner Merkmale untereinander zu nutzen wurden weitere Features aus dem Datensatz zusammengefasst. Die so neu entstandenen Features sind „Bedrooms-per-Room“, „Dist-to-SF“ und „Dist-to-LA“, als Hypothese einer Preissteigerung in Abhängigkeit der relativen Nähe zu den größten Ballungsräumen, sowie „Lat-x-Lon“, als allgemeine geografische Einordnung. Aufgrund der Schwierigkeiten mit schiefen Verteilungen wurde zudem die Population in logarithmischer Form eingebracht. Auch die Einführung dieser Features brachte keine nennenswerte Leistungssteigerung (ca.0.5%) oder Fehlerminderung mit sich. Dennoch haben vor allem die beiden Distance-Features einen Mehrwert für das Projekt, da ihre Feature Importance relativ hoch ist. Wie im Plot Fig. 4 zu sehen, haben die Distancefeatures keine unwesentliche Feature Importance. Das Median Einkommen bleibt jedoch das stärkste Feature für den Datensatz.

**Abbildung 4.** Featureimportance

Obgleich vorallem die Distance-Features keine unwesentliche Feature Importance aufweisen, bleibt das Median Einkommen das stärkste Feature für den Datensatz.

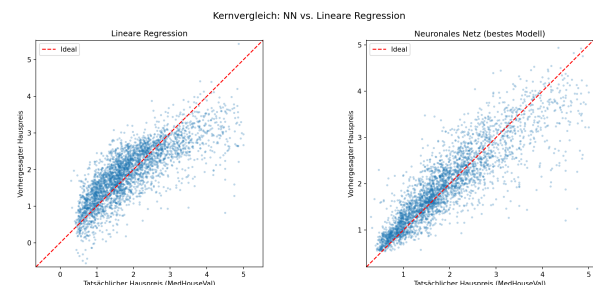
6.3. Gesamtvergleich mit der Linearen Regression

Der Gesamtvergleich zwischen linearer Regression und unserem stärksten Neuronales Netz zeigt eine deutliche Leistungsdivergenz:

Tabelle 3. Vergleich linearer Regression mit Neuronalem Netz

Metrik	Lineare Regression	Neuronales Netz
$R^2 - Score$	0.6326	0.7958 ($\Delta_{Baseline} R^2 = 0.1632$)
MAE	0.4331	0.2895
RMSE	0.5779	0.4308

Der direkte Vergleich zwischen linearer Regression und neuronalen Netzen offenbart ein klassisches Spannungsfeld in der Datenwissenschaft: die Abwägung zwischen Vorhersagegenauigkeit und Interpretierbarkeit. Da bereits ein einfaches neuronales Netz als Minimalnetz einen deutlich besseren Score auf den Testdaten erzielt als die lineare Regression vgl. Fig. 5, lässt sich schließen, dass die Features und ihr Zusammenwirken komplexe, nichtlineare Zusammenhänge aufweisen. Während neuronale Netze diese abbilden können, bedarf es bei der linearen Regression aufwendigen Feature-Engineering, wie etwa durch polynomiale Erweiterungen.

**Abbildung 5.** Lineare Regression vgl. einfaches Neuronales Netz

Das Training eines NN benötigt ein Vielfaches an Rechenzeit und benötigt ein umfangreiches Hyperparameter-Tuning, unter anderem bezüglich der Architektur, Lernrate, Dropout-Rate und Batch-Größe für ein optimales Ergebnis. Wie die Untersuchungen zeigten, neigt beispielsweise ein unreguliertes Modell mit einer [128-64-32]-Neuronenverteilung zu starkem Overfitting. Die Generalisierungsfähigkeit kann jedoch durch den Einsatz von Regularisierung und Dropout erheblich robuster gestaltet werden.

Zudem fehlt den neuronalen Netzen die Interpretierbarkeit fast vollständig. Die lineare Regression hingegen überzeugt durch ihre leicht nachvollziehbaren Koeffizienten.

Die nicht-linearen Zusammenhänge des kalifornischen Hauspreis-Datensatzes lassen sich sehr gut durch ein neuronales Netz beschreiben. Das Optimalergebnis des Random Forest mit Gradient Boosting wurde jedoch noch nicht erreicht. Mögliche Ansätze für noch weitere Optimierungen wäre zum Beispiel der Einsatz von Batch Normalization zur Stabilisierung des Lernprozesses oder die Verwendung eines Powertransformers. Ansätze wie die bereits untersuchte „manuelle Logarithmisierung der Population“ könnten damit automatisiert werden. Dass dies zu Lasten der Interpretierbarkeit geht, wäge im Vergleich zur linearen Regression nicht weiter auf.